Data Structures in Python

A Practical Guide with Explanations, Diagram-Style Notes, Code Examples, and Complexity Analysis

---

Covers: Arrays & Lists, Stacks, Queues, Linked Lists, Trees, Binary Search Trees, Hash Tables, Graphs, and Big-O Analysis

Table of Contents

Chapter 1: Introduction to Data Structures

A **data structure** is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently. Choosing the right data structure can make the difference between a program that runs instantly and one that grinds to a halt as data grows.

Python is an excellent language for learning data structures because it ships with several of them built in (lists, tuples, dictionaries, sets) while still allowing you to build classic structures like linked lists, trees, and graphs from scratch using classes.

Why Data Structures Matter

- **Efficiency:** The right structure reduces time and memory complexity of operations.
- **Organization:** They model relationships in data — sequential, hierarchical, or networked.
- **Foundations:** Almost every algorithm (sorting, searching, pathfinding) relies on a data structure.
- **Interviews and real systems:** Databases, compilers, operating systems, and games are all built on these concepts.

Abstract Data Types vs. Implementations

An **Abstract Data Type (ADT)** defines *what* operations a structure supports (e.g., a Stack supports push and pop) without specifying *how* it is implemented. A concrete data structure is one particular implementation of that ADT. For example, a Stack ADT can be implemented using a Python list or a linked list — both satisfy the same interface but may have different performance trade-offs.

Time and Space Complexity (Big-O Notation)

Throughout this guide we describe the performance of operations using **Big-O notation**, which expresses how the time or memory an operation needs grows relative to the size of the input, n. Common complexities, from fastest to slowest growth:

Notation	Name	Example
O(1)	Constant	Accessing a list element by index
O(log n)	Logarithmic	Binary search in a sorted list
O(n)	Linear	Searching an unsorted list
O(n log n)	Linearithmic	Efficient sorting (merge sort)
O(n^2)	Quadratic	Nested loops comparing all pairs

With that foundation in place, let's explore each data structure in detail, starting with the most fundamental one: the array.

Chapter 2: Arrays and Python Lists

An **array** is a collection of elements stored in contiguous memory locations, accessed using an index. In Python, the built-in **list** type behaves like a dynamic array: it can grow and shrink automatically, and it can hold elements of different types.

Key Characteristics

- Elements are ordered and indexed starting at 0.
- Accessing an element by index is O(1) because the memory address can be computed directly.
- Python lists are *dynamic arrays*: when they run out of space, Python allocates a larger block and copies the elements over automatically.
- Insertion or deletion at the end is usually O(1); at the beginning or middle it is O(n) because remaining elements must shift.

Common Operations

```
# Creating a list
numbers = [4, 8, 15, 16, 23, 42]

# Accessing an element (O(1))
print(numbers[2])          # 15

# Appending to the end (O(1) amortized)
numbers.append(99)

# Inserting at a specific position (O(n))
numbers.insert(0, 1)       # shifts every element right

# Removing an element by value (O(n))
numbers.remove(8)

# Removing by index (O(n) unless it's the last element)
numbers.pop(0)

# Slicing (creates a new list, O(k) where k is slice length)
sub = numbers[1:4]

# Searching for a value (O(n))
exists = 42 in numbers

# Length (O(1))
print(len(numbers))
```

List Comprehensions

Python's list comprehensions provide a concise, readable way to build new lists from existing iterables, often replacing explicit for-loops.

```
squares = [x * x for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
```

```
matrix_flat = [val for row in matrix for val in row]
```

When to Use Lists

Use Python lists when you need an ordered, index-accessible collection and mostly append or read from the end. Avoid frequent insertions/deletions at the front of large lists — consider a **deque** (from the collections module) instead, which supports O(1) operations at both ends.

```
from collections import deque
dq = deque([1, 2, 3])
dq.appendleft(0)   # O(1)
dq.popleft()       # O(1)
```

Operation	Average Case	Worst Case
Index access	O(1)	O(1)
Append (end)	O(1)	O(n) on resize
Insert (start/middle)	O(n)	O(n)
Search (unsorted)	O(n)	O(n)
Delete by value	O(n)	O(n)

Chapter 3: Stacks

A **stack** is a linear data structure that follows the **LIFO** principle: *Last In, First Out*. Think of a stack of plates — you add (push) plates to the top and remove (pop) from the top as well. You cannot access the plates underneath without first removing the ones above.

Core Operations

- **push(item):** add an item to the top of the stack — O(1)
- **pop():** remove and return the top item — O(1)
- **peek() / top():** view the top item without removing it — O(1)
- **is_empty():** check whether the stack has any elements — O(1)

Implementing a Stack with a Python List

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)          # add to the end (top)

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._items.pop()          # remove from the end (top)

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def __len__(self):
        return len(self._items)


s = Stack()
s.push(10)
s.push(20)
s.push(30)
print(s.pop())    # 30
print(s.peek())   # 20
```

Notice that using append and pop (both without an index) on a Python list is efficient because both operations act on the end of the list, which is O(1).

Real-World Uses of Stacks

- Undo/redo functionality in text editors.
- Function call management — the *call stack* that every program uses.
- Balanced parentheses / syntax checking in compilers.
- Backtracking algorithms (maze solving, depth-first search).
- Browser back-button history.

Example: Checking Balanced Parentheses

```
def is_balanced(expression):
    stack = []
    pairs = {')': '(', ']': '[', '}': '{'}

    for char in expression:
        if char in '([{':
            stack.append(char)
        elif char in ')]}':
            if not stack or stack.pop() != pairs[char]:
                return False

    return len(stack) == 0

print(is_balanced("{[()()]}"))   # True
print(is_balanced("{[(])}"))     # False
```

Chapter 4: Queues

A **queue** is a linear data structure that follows the **FIFO** principle: *First In, First Out*. Imagine a line of people waiting to buy tickets — the first person to join the line is the first one served.

Core Operations

- **enqueue(item):** add an item to the back of the queue — O(1)
- **dequeue():** remove and return the item at the front — O(1) with the right structure
- **front():** view the front item without removing it — O(1)
- **is_empty():** check whether the queue has elements — O(1)

A common beginner mistake is implementing a queue with a plain Python list and calling list.pop(0) to dequeue. This is O(n) because every remaining element must shift left. Use collections.deque instead, which is implemented as a doubly linked list and supports O(1) operations at both ends.

Implementing a Queue with collections.deque

```
from collections import deque

class Queue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, item):
        self._items.append(item)          # add to the back

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self._items.popleft()      # remove from the front

    def front(self):
        if self.is_empty():
            raise IndexError("front from empty queue")
        return self._items[0]

    def is_empty(self):
        return len(self._items) == 0


q = Queue()
q.enqueue("A")
q.enqueue("B")
q.enqueue("C")
print(q.dequeue())   # "A"
print(q.front())     # "B"
```

Variations of Queues

- **Circular queue:** reuses freed space at the front by wrapping around, common in fixed-size buffers.

- **Priority queue:** elements are dequeued based on priority, not arrival order. Python provides the heapq module for this.
- **Double-ended queue (deque):** allows insertion and removal from both ends.

Example: Priority Queue with heapq

```
import heapq

tasks = []
heapq.heappush(tasks, (2, "write report"))
heapq.heappush(tasks, (1, "fix critical bug"))
heapq.heappush(tasks, (3, "reply to email"))

while tasks:
    priority, task = heapq.heappop(tasks)
    print(priority, task)

# Output order:
# 1 fix critical bug
# 2 write report
# 3 reply to email
```

Real-World Uses of Queues

- Task scheduling and print job management.
- Breadth-first search (BFS) in graphs and trees.
- Handling requests in web servers (request queues).
- Buffering data streams (e.g., video streaming, keyboard input).

Chapter 5: Linked Lists

A **linked list** is a linear collection where each element, called a **node**, holds its own data plus a reference (pointer) to the next node. Unlike arrays, linked lists do not require contiguous memory, which makes insertion and deletion very efficient — as long as you already have a reference to the relevant node.

Singly Linked List Node

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Building a Singly Linked List

```
class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        """Add a node to the end of the list. O(n) without a tail pointer."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def prepend(self, data):
        """Add a node to the front of the list. O(1)."""
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def delete(self, target):
        """Delete the first node containing target. O(n)."""
        current = self.head
        previous = None
        while current:
            if current.data == target:
                if previous is None:
                    self.head = current.next
                else:
                    previous.next = current.next
                return True
            previous = current
            current = current.next
        return False

    def __str__(self):
        values = []
        current = self.head
        while current:
```

```
            values.append(str(current.data))
            current = current.next
        return " -> ".join(values) + " -> None"


ll = LinkedList()
ll.append(1)
ll.append(2)
ll.append(3)
ll.prepend(0)
print(ll)          # 0 -> 1 -> 2 -> 3 -> None
ll.delete(2)
print(ll)          # 0 -> 1 -> 3 -> None
```

Singly vs. Doubly Linked Lists

In a **singly linked list**, each node points only to the next node, so traversal is one-directional. In a **doubly linked list**, each node also holds a reference to the *previous* node, which allows traversal in both directions and O(1) removal of a node when you already have a reference to it, without needing to track the previous node manually.

```
class DoublyNode:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None
```

Arrays vs. Linked Lists

Aspect	Array / Python List	Linked List
Access by index	O(1)	O(n)
Insert/delete at front	O(n)	O(1)
Insert/delete at end	O(1) amortized	O(1) with tail pointer
Memory layout	Contiguous	Scattered (nodes + pointers)
Extra memory per element	None	One or two pointers

Real-World Uses

- Implementing stacks, queues, and deques efficiently.
- Undo history where nodes represent states.
- Music/playlist "next" and "previous" navigation (doubly linked).
- Memory allocators and operating system structures.

Chapter 6: Trees and Binary Search Trees

A **tree** is a hierarchical data structure consisting of **nodes** connected by edges, starting from a single **root** node. Each node may have zero or more **children**, and a node with no children is called a **leaf**. Trees model hierarchical relationships such as file systems, organizational charts, and parsed expressions.

Key Terminology

- **Root:** the top-most node of the tree.
- **Parent / Child:** a node directly connected above/below another.
- **Leaf:** a node with no children.
- **Height:** the number of edges on the longest path from the root to a leaf.
- **Depth:** the number of edges from the root to a given node.

Binary Trees

A **binary tree** is a tree in which every node has at most two children, commonly referred to as the **left** and **right** child.

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Binary Search Trees (BST)

A **Binary Search Tree** is a binary tree with an important ordering property: for every node, all values in its left subtree are smaller, and all values in its right subtree are greater. This property makes searching, insertion, and deletion efficient — O(log n) on average for a balanced tree.

```
class BST:
    def __init__(self):
        self.root = None

    def insert(self, value):
        self.root = self._insert(self.root, value)

    def _insert(self, node, value):
        if node is None:
            return TreeNode(value)
        if value < node.value:
            node.left = self._insert(node.left, value)
        elif value > node.value:
            node.right = self._insert(node.right, value)
        return node   # duplicates are ignored here

    def search(self, value):
        return self._search(self.root, value)
```

```
    def _search(self, node, value):
        if node is None:
            return False
        if node.value == value:
            return True
        elif value < node.value:
            return self._search(node.left, value)
        else:
            return self._search(node.right, value)

    def inorder(self):
        """Returns values in sorted order."""
        result = []
        self._inorder(self.root, result)
        return result

    def _inorder(self, node, result):
        if node:
            self._inorder(node.left, result)
            result.append(node.value)
            self._inorder(node.right, result)


tree = BST()
for v in [50, 30, 70, 20, 40, 60, 80]:
    tree.insert(v)

print(tree.search(40))   # True
print(tree.search(99))   # False
print(tree.inorder())    # [20, 30, 40, 50, 60, 70, 80]
```

Tree Traversal Strategies

- **Inorder** (left, node, right): visits nodes in sorted order for a BST.
- **Preorder** (node, left, right): useful for copying a tree or prefix expressions.
- **Postorder** (left, right, node): useful for deleting a tree or postfix expressions.
- **Level-order (BFS):** visits nodes level by level using a queue.

```
from collections import deque

def level_order(root):
    if root is None:
        return []
    result = []
    queue = deque([root])
    while queue:
        node = queue.popleft()
        result.append(node.value)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return result
```

Balanced vs. Unbalanced Trees

If values are inserted in already-sorted order (e.g., 1, 2, 3, 4, 5) a BST degenerates into a structure that behaves like a linked list, with O(n) operations. Self-balancing trees such as **AVL trees** and **Red-Black trees** automatically re-arrange nodes during insertion and deletion to guarantee O(log n) operations in the worst case.

Operation	Balanced BST	Unbalanced (worst case)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

Real-World Uses of Trees

- File systems (folders and subfolders).
- Databases and search indexes (B-trees).
- HTML/XML/DOM document structure.
- Decision trees in machine learning.
- Autocomplete and spell-checking (tries, a specialized tree).

Chapter 7: Hash Tables (Dictionaries)

A **hash table** stores key-value pairs and uses a **hash function** to compute an index (a "bucket") into an array where each value is stored. This allows average-case O(1) lookup, insertion, and deletion — dramatically faster than scanning a list. Python's built-in **dict** (and **set**) are implemented as hash tables.

How Hashing Works

- A hash function converts a key (e.g., a string) into an integer.
- That integer is reduced (via modulo) to fit within the table's current size, giving a bucket index.
- The key-value pair is stored at that bucket.
- When two different keys hash to the same bucket, a **collision** occurs, which is resolved through techniques such as *chaining* (each bucket holds a small list) or *open addressing* (probing for the next free slot).

Using Python Dictionaries

```
# Creating a dictionary
student_grades = {"Alice": 92, "Bob": 85, "Carla": 78}

# Insertion / update — O(1) average
student_grades["David"] = 88

# Lookup — O(1) average
print(student_grades["Bob"])          # 85

# Checking existence — O(1) average
print("Alice" in student_grades)      # True

# Deletion — O(1) average
del student_grades["Carla"]

# Iterating over keys, values, or both
for name, grade in student_grades.items():
    print(name, grade)

# Using .get() to avoid KeyError
print(student_grades.get("Zoe", "not found"))
```

Implementing a Simple Hash Table from Scratch

The following implementation demonstrates the underlying mechanics using **chaining** to handle collisions — each bucket is a small list of key-value pairs.

```
class HashTable:
    def __init__(self, size=8):
        self.size = size
        self.buckets = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size
```

```
    def put(self, key, value):
        index = self._hash(key)
        bucket = self.buckets[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)   # update existing key
                return
        bucket.append((key, value))        # insert new key

    def get(self, key):
        index = self._hash(key)
        bucket = self.buckets[index]
        for k, v in bucket:
            if k == key:
                return v
        raise KeyError(key)

    def remove(self, key):
        index = self._hash(key)
        bucket = self.buckets[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]
                return
        raise KeyError(key)


table = HashTable()
table.put("apple", 3)
table.put("banana", 6)
print(table.get("banana"))   # 6
```

Sets: Hash Tables Without Values

A Python **set** is essentially a hash table that stores only keys (no associated values), useful for membership testing and removing duplicates in O(n) overall time.

```
unique_ids = set([1, 2, 2, 3, 3, 3])
print(unique_ids)             # {1, 2, 3}
print(2 in unique_ids)        # O(1) average
```

Operation	Average Case	Worst Case
Insert	O(1)	O(n) with many collisions
Lookup	O(1)	O(n) with many collisions
Delete	O(1)	O(n) with many collisions

Real-World Uses

- Caching and memoization.
- Database indexing.
- Counting word frequencies or deduplicating data.

- Implementing sets, symbol tables in compilers, and JSON-like objects.

Chapter 8: Graphs

A **graph** is a set of **vertices** (nodes) connected by **edges**. Graphs generalize trees: while a tree cannot contain cycles and has exactly one path between any two nodes, a graph may contain cycles and multiple paths, and does not require a single root.

Types of Graphs

- **Directed vs. Undirected:** edges may have direction (A → B) or be bidirectional (A — B).
- **Weighted vs. Unweighted:** edges may carry a cost or distance value.
- **Cyclic vs. Acyclic:** a graph with no cycles is called a DAG (Directed Acyclic Graph) when directed.

Representing Graphs in Python

The most common representation is an **adjacency list**, typically built with a dictionary mapping each node to a list (or set) of its neighbors.

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C", "E"],
    "E": ["D"],
}
```

Adjacency List vs. Adjacency Matrix

Aspect	Adjacency List	Adjacency Matrix
Space	O(V + E)	O(V^2)
Check if edge exists	O(degree of vertex)	O(1)
Iterate over neighbors	O(degree of vertex)	O(V)
Best for	Sparse graphs	Dense graphs

Breadth-First Search (BFS)

BFS explores a graph level by level using a queue, and is the natural choice for finding the **shortest path** in an unweighted graph.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []

    while queue:
```

```
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order

print(bfs(graph, "A"))   # ['A', 'B', 'C', 'D', 'E']
```

Depth-First Search (DFS)

DFS explores as far as possible along a branch before backtracking, typically implemented recursively or with an explicit stack. It is useful for detecting cycles, topological sorting, and exploring all paths.

```
def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
        order = []
    visited.add(start)
    order.append(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)
    return order

print(dfs(graph, "A"))   # ['A', 'B', 'D', 'C', 'E']
```

BFS vs. DFS

Aspect	BFS	DFS
Data structure used	Queue	Stack (or recursion)
Finds shortest path?	Yes (unweighted graphs)	No guarantee
Memory usage	Can be high (wide graphs)	Can be high (deep graphs)
Typical use	Shortest path, level order	Cycle detection, topological sort

Real-World Uses of Graphs

- Social networks (people as nodes, friendships as edges).
- Maps and navigation systems (shortest-path algorithms like Dijkstra's).
- Web page linking structure (used by search engine ranking algorithms).
- Dependency resolution (package managers, build systems).
- Recommendation engines.

Chapter 9: Big-O Cheat Sheet and Final Comparison

This chapter consolidates the performance characteristics of every structure covered in this guide, so you can quickly compare them when designing a solution.

Structure	Access	Search	Insert	Delete
Array / Python List	O(1)	O(n)	O(n)*	O(n)
Stack	O(n)	O(n)	O(1)	O(1)
Queue (deque)	O(n)	O(n)	O(1)	O(1)
Singly Linked List	O(n)	O(n)	O(1)**	O(1)**
Balanced BST	O(log n)	O(log n)	O(log n)	O(log n)
Hash Table (dict/set)	N/A	O(1) avg	O(1) avg	O(1) avg
Graph (adjacency list)	O(1) node	O(V+E)	O(1)	O(degree)

** O(1) amortized when appending to the end. ** O(1) only when you already hold a reference to the relevant node.*

Chapter 10: Choosing the Right Data Structure

A practical guide to picking a data structure starts with the operations your program performs most often. Use the questions below as a decision checklist.

Decision Guide

- **Need fast index-based access and mostly add/read at the end?** Use a Python **list**.
- **Need Last-In-First-Out behavior (undo, backtracking)?** Use a **stack** (a list works fine).
- **Need First-In-First-Out behavior (scheduling, BFS)?** Use a **queue** (collections.deque).
- **Need fast insertion/removal at arbitrary positions and don't need random access?** Consider a **linked list**.
- **Need to maintain sorted order with fast search, insert, and delete?** Use a **balanced binary search tree**.
- **Need the fastest possible key-based lookup and don't care about order?** Use a **hash table** (dict or set).
- **Need to model relationships or connections between entities?** Use a **graph**.

Final Thoughts

Mastering data structures is less about memorizing code and more about developing intuition for *trade-offs*: every structure sacrifices something (memory, ordering, or flexibility) to gain speed somewhere else. As you write more Python programs, try to identify which operations dominate your workload and let that guide your choice of structure — that habit alone will make your code faster and more scalable.

End of guide — happy coding!